



DEF CON – Workshop

LVCC - L2 - N254, Saturday 09:00-13:00

Eliminating Bug Classes at Scale: Leveraging Browser Features for Proactive Defense

Javan Rasokat
Senior Application Security Specialist,
DevOps Security at Sage

Agenda

- Common web security flaws
- OWASP Proactive Controls
- Case Study: Scaling and Measuring the adoption of modern web standards by Google
- Modern browser security features for defense in depth
- Hands-on activities: Securing “Vegas Nightlife App”



\$ whoami



@javan rasokat

- Senior Application Security Specialist, DevOps Security at **Sage**
- Lecturer for Secure Coding
- Passionate about web security, Raspberry Pi, and home automation
- I used to do development, then I started breaking stuff, now I am focusing on scaling app sec and building stuff again

Common web security flaws

MITRE - CWE Top 25 (2024)



2024 CWE Top 25 Most Dangerous Software Weaknesses

Top 25 Home | Share via: ▼ | View in table format | Key insights | Methodology

1	Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting') CWE-79 CVEs in KEV: 3 Rank Last Year: 2 (up 1) ▲
2	Out-of-bounds Write CWE-787 CVEs in KEV: 18 Rank Last Year: 1 (down 1) ▼
3	Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') CWE-89 CVEs in KEV: 4 Rank Last Year: 3
4	Cross-Site Request Forgery (CSRF) CWE-352 CVEs in KEV: 0 Rank Last Year: 9 (up 5) ▲
5	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal') CWE-22 CVEs in KEV: 4 Rank Last Year: 8 (up 3) ▲
6	Out-of-bounds Read CWE-125 CVEs in KEV: 3 Rank Last Year: 7 (up 1) ▲
7	Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection') CWE-78 CVEs in KEV: 5 Rank Last Year: 5 (down 2) ▼
8	Use After Free CWE-416 CVEs in KEV: 5 Rank Last Year: 4 (down 4) ▼
9	Missing Authorization CWE-862 CVEs in KEV: 0 Rank Last Year: 11 (up 2) ▲
10	Unrestricted Upload of File with Dangerous Type CWE-434 CVEs in KEV: 0 Rank Last Year: 10

Source: https://cwe.mitre.org/top25/archive/2024/2024_cwe_top25.html

Most common platform vulnerability	Platform Average	 Financial Services	 Government	 Telecoms	 Retail & E-commerce	 Transportation	 Media & Entertainment	 Computer Software	 Internet & Online Services	 Crypto & Blockchain	 Travel & Hospitality
Cross-Site Scripting (XSS)	20%	19%	40%	15%	19%	34%	21%	19%	17%	7%	27%
Information Disclosure	10%	11%	13%	12%	16%	13%	9%	9%	13%	7%	10%
Improper Access Control	9%	12%	5%	10%	8%	8%	10%	13%	13%	10%	8%
Misconfiguration	6%	7%	2%	6%	8%	5%	12%	8%	7%	6%	9%
Insecure Direct Object Reference (IDOR)	6%	7%	1%	12%	9%	6%	8%	6%	8%	2%	6%
Improper Authentication	2%	3%	2%	7%	3%	2%	3%	3%	3%	4%	2%
Privilege Escalation	2%	3%	1%	3%	2%	2%	4%	4%	3%	2%	4%
Open Redirect	2%	4%	1%	2%	4%	4%	4%	2%	2%	2%	4%
Business Logic Errors	2%	2%	0%	2%	3%	1%	3%	2%	3%	10%	1%
SQL Injection	2%	2%	7%	4%	3%	4%	1%	1%	1%	0%	2%

**...why Do These
Vulnerabilities
Persist?**

Application Security Anti-Patterns

- Constant Emgerence of Issues
- Reactive, Not Proactive
- Never-Ending Cycle
- Stressful and Resource-Intensive
- Limited Focus on Root Causes



Firefighting Instead of Building Resilience

- Reactive measures address **symptoms, not root causes**
- **Eliminate bug classes** with modern security controls
- Build resilience through **secure-by-default** principles
- Scale security **with automation** and **proactive strategies**
- Empower teams to adopt **defence-in-depth** at scale



OWASP Top 10 Proactive Controls (2024 Version)

- C1: Implement Access Control
- C2: Use Cryptography to Protect Data
- C3: Validate all Input & Handle Exceptions
- C4: Address Security from the Start
- C5: Secure By Default Configurations
- C6: Keep your Components Secure
- C7: Secure Digital Identities
- **C8: Leverage Browser Security Features – 🌟 NEW 2024 🌟**
- C9: Implement Security Logging and Monitoring
- C10: Stop Server Side Request Forgery



<https://top10proactive.owasp.org/>

Backwards Compatibility and the Evolution of Browser Security

- **Backwards compability is crucial:**
 - Browsers cannot break the web by fixing security issues (“design flaws”) without consideration for existing websites.
- **Opt-in mechanisms were introduced:**
 - Features like Content Security Policy (CSP).
- **Secure by design approaches are emerging:**
 - Redirecting HTTP to HTTPS
 - Changing the SameSite attribute to Lax if undefined
 - Preventing some cases of mXSS (mutation-based)
- Browsers are finding a balance between security and compatibility, with a focus on protecting users while minimizing disruptions.

Google Research: Security Signals

Google Research

Who we are ▾

Research areas ▾

Our work ▾

Programs & events ▾

Careers

Blog

[Home](#) > [Publications](#) >

Security Signals: Making Web Security Posture Measurable At Scale

[Michele Spagnuolo](#) · David Dworken · Artur Janc · Santiago (Sal) Díaz · [Lukas Weichselbaum](#) · (2024) (to appear)

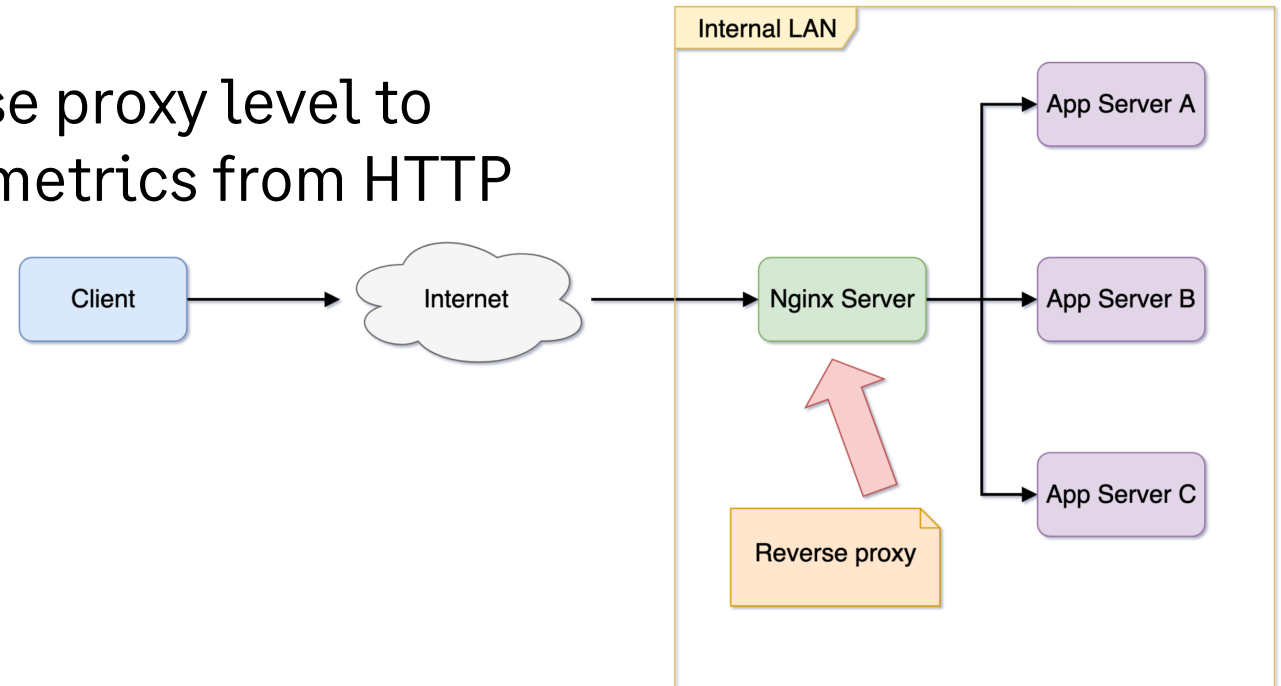
Download

Google Scholar

Copy Bibtex

Security Signals: Enabling Scalable Security

- **Challenge:**
 - Adopting, but also measuring the adoption in a diverse, large-scale web ecosystem.
- **Solution:**
 - Inspect traffic on a reverse proxy level to collect runtime security metrics from HTTP traffic.



Synthetic Signals: Custom Metrics for Scalable Security

- **What Are Synthetic Signals?**
 - Custom HTTP headers used to expose security properties at runtime.
 - **Examples:** CSRF checks, safe templating systems, server-side isolation policies.
- **Key Advantages:**
 - Technology-Agnostic
 - Works across frameworks, programming languages, and infrastructure.
 - Enhanced Visibility
 - Provides granular insights into application behaviour and defences.
 - Low Overhead
 - Added at the reverse proxy layer with minimal performance impact.
- **Impact:**
 - Identifies unsafe patterns on all endpoints at scale.
 - Facilitates automated remediation through actionable insights.
 - Enables targeted deployment of security mechanisms like Trusted Types and CSP.

Synthetic Signals : Analysing Key Metrics

- **Framework**
 - Differentiation between hardened and safe-by-default vs. legacy frameworks.
- **CSP**
 - The presense of a strict CSP Policy.
- **CSRF**
 - The presence of any CSRF protections.
- **Sec-Fetch**
 - The presense of server-side isolation policies.

...

Measuring posture with Scorecards

 Web Security Portal

Last data is from 2 days ago  

Search by Team, Product Area, Host, GFE Service, or G3 Project

com

Strict CSP 

1/1 projects protected

Adoption steps for

> Core > Experience >

> Account Experience > Account Life Cycle > Frontend Infrastructure >

.identity.IdentityFrontendUIServer (Web):

 File Tracking Bug

 Deploy Report-Only

 Review Violations

 Enforce Policy

 Validate Fix

Once the Web Security Portal shows your service as protected, mark the bug () as fixed and verified, and celebrate!

Note: It can take up to two days for changes to reach production and be reflected in the Web Security Portal.

Project	Framework	Priority/Tier 	Protected req/day	Deployment	Tools	Bug
Frontend Infrastructure > identity.IdentityFrontendUIServer	Web	Tier 0			   	 File

> Trusted Types 

1/1 projects protected

> Fetch Metadata Resource Isolation Policy 

1/1 projects protected

> Framing Controls and Clickjacking Protection 

1/1 projects protected

> Cross Origin Opener Policy 

1/1 projects protected

> Fetch Metadata Framing Isolation Policy 

1/1 projects protected

> Hostname Validation 

1/1 projects protected

> 3rd Party Script Blocking via Allowlist CSP 

1/1 projects protected

> Origin-Scoped Cookies 

1/1 projects protected

> Strict Transport Security 

1/1 projects protected

> Cross Origin Resource Policy 

1/1 projects protected

Source: Google, **Security Signals: Making Web Security Posture Measurable At Scale**

Browser Security Features

– for building secure-by-default apps

Cross-Site Request Forgery (CSRF)

```

```

CSRF Attack Example - Forms

Auto-submits a POST request to a trusted site

```
<!-- Malicious Page -->
<form action="https://trusted-site.com/updatePassword" method="POST">
  <input type="hidden" name="password" value="newpassword123">
  <input type="hidden" name="confirmPassword" value="newpassword123">
  <button type="submit">Click me</button>
</form>

<script>
  // Auto-submit the form without user interaction
  document.querySelector('form').submit();
</script>
```

CSRF Attack Example - Image Tags or Script Tags

A malicious page loads an image or script tag to trigger a sensitive GET request.

```
<!-- Malicious Page -->  
  
<!-- Or -->  
<script src="https://trusted-site.com/triggerSensitiveAction"></script>
```

CSRF Attack Example – XHR Requests

Misconfigured Cross-Origin-Resource-Sharing (CORS) policies allow unauthorised cross-origin XHR requests.

```
// Malicious Page
fetch("https://trusted-site.com/api/transfer", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({ amount: 1000, to: "attacker_account" })
})
.then(response => console.log("Request sent:", response.status));
```

CSRF – Prevention Techniques “Catalog”

- **Your responsibility (or built-in by a framework):**
 - Token Synchroniser Pattern (“Anti-CSRF tokens”)
 - Double-Submit Cookie Pattern
- **Browser native features***
 - Same-Origin Policy (as a foundational basis)
 - check for CORS misconfigs
 - Cookies “SameSite”-attribute
 - Sec-Fetch Metadata 🌟 **NEW** 🌟

For a more comprehensive overview see [OWASP CSRF Prevention Cheatsheet](#).

(* some are considered defense-in-depth and not “enough” by themselves to fully mitigate all CSRF scenarios)

Fetch- Metadata Headers

```
if (headers['sec-fetch-site'] === 'same-origin' || headers['sec-fetch-site'] === 'same-site') {  
  return next();  
}
```


Curl vs. HTTP GET in the browser

```
curl example.com -v
```

Output:

```
GET / HTTP/1.1
```

```
Host: example.com
```

```
User-Agent: curl/8.7.1
```

```
Accept: */*
```

► GET https://example.com/

Status	200 ?
Version	HTTP/3
Transferred	1,09 kB (1,26 kB size)
Request Priority	Highest
DNS Resolution	DNS over HTTPS

► Response Headers (439 B)

▼ Request Headers (530 B)

- ? Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
- ? Accept-Encoding: gzip, deflate, br, zstd
- ? Accept-Language: en-US,en;q=0.8,de-DE;q=0.5,de;q=0.3
- Alt-Used: example.com
- ? Cache-Control: no-cache
- ? Connection: keep-alive
- ? DNT: 1
- ? Host: example.com
- ? Pragma: no-cache
- ? Priority: u=0, i
- ? Sec-Fetch-Dest: document
- ? Sec-Fetch-Mode: navigate
- ? Sec-Fetch-Site: cross-site
- ? TE: trailers

Introducing Fetch Metadata Request Headers

`https://site.example`

`fetch("https://site.example/foo.json")`

```
GET /foo.json
Host: site.example
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
```

`https://evil.example`

``

```
GET /foo.json
Host: site.example
Sec-Fetch-Site: cross-site
Sec-Fetch-Mode: no-cors
Sec-Fetch-Dest: image
```

Source: <https://web.dev/articles/fetch-metadata>

Time to practice!

**Build your own
protection.**

Securing Las Vegas Nightlife App

Challenge 1



<https://github.com/JavanXD/LasVegasNightlifeApp-Workshop>

```
<meta http-equiv="Content-Security-Policy"  
  content="  
    default-src 'self';  
    script-src 'self' 'nonce-123abc' 'strict-dynamic';  
    object-src 'none';  
    style-src 'self' 'nonce-123abc';  
    img-src 'self' data:;  
    connect-src 'self';  
    font-src 'self';  
    frame-ancestors 'none';  
    base-uri 'self';  
    form-action 'self';  
    block-all-mixed-content;  
    upgrade-insecure-requests;  
    report-uri https://example.com/csp-report;  
">
```

Content Security Policy (CSP)

Building Scalable Defences with Content Security Policy (CSP)

- **What is CSP?**
 - A security standard to prevent XSS and other injection attacks.
 - Allows you to control which resources (scripts, styles, etc.) can be loaded and executed.
- **Why CSP Matters at Scale:**
 - Protects against the most common web vulnerabilities (e.g. XSS).
 - Ensures consistent security across multiple products and teams.
 - Centralised Monitoring allows to react on policy violations and to identify supply-chain risks like “polyfills.io”.
- **Challenges:**
 - Complex policies across dynamic apps.
 - Maintaining balance between security and compatibility.
 - Automating policies and reporting for multiple applications.

Trade-offs in CSP Implementation

Strict Policies

- Strong XSS protection.
- Potential to break legitimate functionality.

Lenient Policies

- Strong adoption, fewer disruptions.
- Reduced protection; may allow unsafe practices.

Finding the Balance:

Start with monitoring mode (Content-Security-Policy-Report-Only) to identify violations before enforcing strict rules.



The “Vegas Nightlife App” – it’s current CSP (1/2)



Las Vegas Nightlife App

High Stakes Guest List – Las Vegas Edition

 Join the VIP Guest List

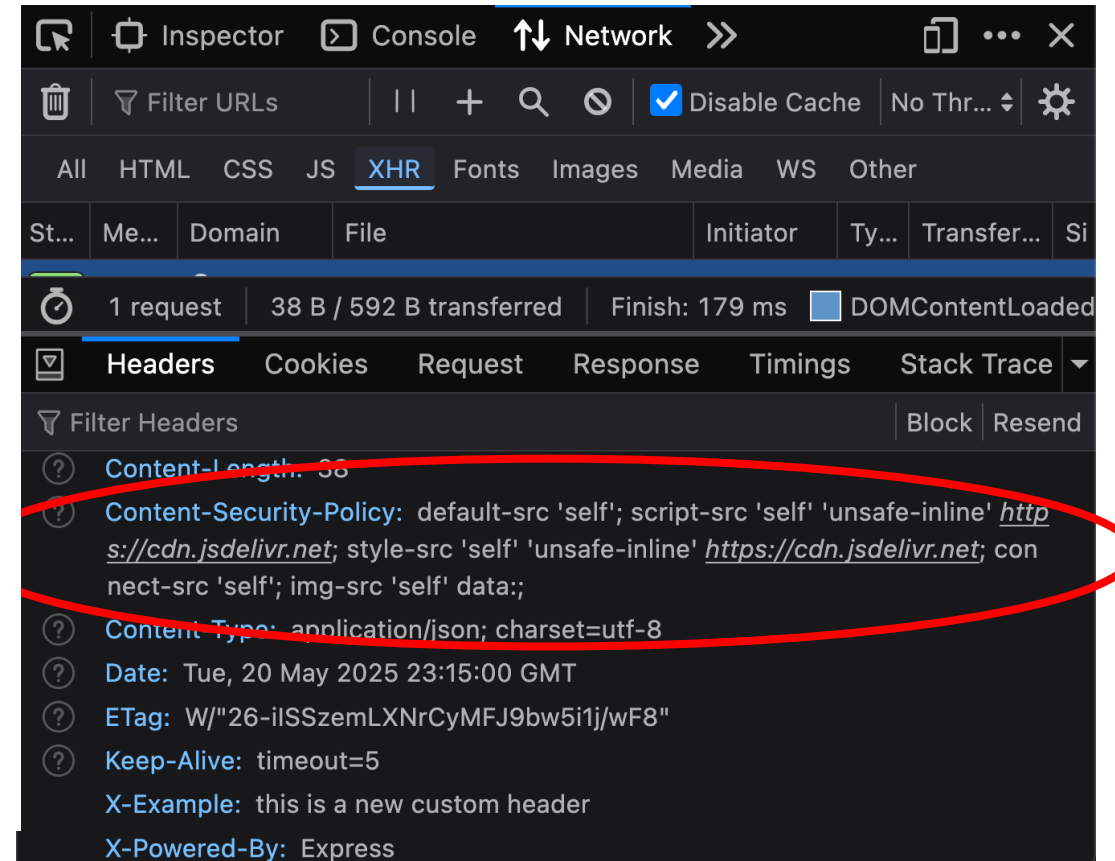
Your Name

John Doe

 Event/Game Selection

High Rollers Poker

 Join the Table



The “Vegas Nightlife App” – it’s current CSP (2/2)

default-src 'self':

Restricts all unspecified resource types to the same origin.

script-src 'self' 'unsafe-inline' https://cdn.jsdelivr.net:

Allows inline scripts (e.g., `<script>` tags and `onerror` attributes), enabling potential XSS attacks.

Allows scripts from the trusted CDN `https://cdn.jsdelivr.net` (used for Bootstrap).

style-src 'self' 'unsafe-inline' https://cdn.jsdelivr.net:

Allows inline styles and stylesheets from the trusted CDN.

connect-src 'self':

Restricts fetch and XHR requests to the same origin.

img-src 'self' data::

Allows images from the same origin and inline images encoded as data: URLs.

Nonces, hashes – makes a “strict” CSP

Content-Security-Policy

```
object-src 'none'; base-uri 'none';
script-src 'nonce-r4nd0m' 'strict-dynamic';
```

Execute only scripts with the correct *nonce* attribute

✓ `<script nonce="r4nd0m">kittens()</script>`
 ✗ `<script nonce="other-value">evil()</script>`

Trust scripts added by already trusted code

✓ `<script nonce="r4nd0m">`
 `var s = document.createElement('script');`
 `s.src = "/path/to/script.js";`
 ✓ `document.head.appendChild(s);`
 `</script>`

Source: <https://web.dev/articles/strict-csp>

Securing Las Vegas Nightlife App

Challenge 2 & 3



<https://github.com/JavanXD/LasVegasNightlifeApp-Workshop>

CSP at Scale – Balancing Security and Automation

- **Key Challenges**
 - Managing policies across multiple products.
 - Balancing strictness with compatibility.
 - Handling dynamic content and modern frameworks.
- **Why Automation Matters**
 - Consistency across teams.
 - Reduced manual effort.
 - Scalable reporting and continuous improvement.

Scaling CSP in Five Steps

Steps to Implement CSP at Scale

- *Step 1: Start with a Report-Only Policy*
- *Step 2: Create a Secure and Practical CSP Policy*
- *Step 3: Automate for Consistency and Scale*
- *Step 4: Use Reporting for Continuous Improvement*
- *Step 5: Enforce and Monitor*



Step 1: Monitor Before Enforcing

- Why Report-Only?
 - Identify potential violations **without breaking functionality**.
 - Gather data on **which resources** are blocked.
- How to Implement:
 - Use the Content-Security-Policy-**Report-Only** header.
 - Configure a **report-uri to send violation** reports to a central endpoint.

Step 2: Create a Secure and Practical CSP Policy

- Best Practices:
 - **Use nonces or hashes** instead of unsafe-inline.
 - Limit script and resource sources to **trusted domains**.
 - Avoid wildcard domains (*) whenever possible.
- Iterative Refinement:
 - Start strict and test for compatibility issues.
 - Adjust policy based on violation reports.

Step 3: Automate Policy Management

- Automation Tools:
 - CSP Builders: Automate policy generation in CI/CD pipelines.
 - Static Analysis: Scan code for unsafe practices and recommend CSP adjustments.
- Centralised Management:
 - Use templates to standardise policies across teams.
 - Enforce templates via CI/CD pipelines.
- Dynamic Content:
 - Generate policies dynamically for SPAs and dynamic frameworks.

Step 4: Analyse and Refine with Reporting

- Setup Reporting:
 - Configure report-uri or report-to endpoints for all applications.
 - Centralise reports using tools like Kibana or Splunk.
- Insights to Gather:
 - Most frequently blocked resources.
 - Anomalies or trends in violations.
- Actionable Improvements:
 - Adjust policies based on real-world usage.
 - Detect potential attacks or misconfigurations early.

Step 5: Enforce and Monitor

- Switch to Enforcement Mode:
 - Once confident in your refined policy, enable Content-Security-Policy.
- Continuous Monitoring:
 - Keep report-uri active for ongoing violations.
 - Automate alerts for anomalies.
- Iterate:
 - Adjust policies as applications evolve.
 - Incorporate new standards or requirements.

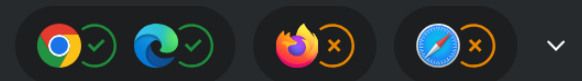
Trust-Types

CSP: require-trusted-types-for

<meta http-equiv="Content-Security-Policy" content="require-trusted-types-for 'script'">



Limited availability



Experimental: This is an experimental technology.

Check the [Browser compatibility table](#) carefully before using this in production.

Trust Types - Concept

Key Concept

Trusted Types prevent **DOM-based XSS** by controlling how HTML is added to the DOM.

Only **trusted, sanitised content** can be used with risky APIs like:

- `innerHTML`
- `outerHTML`
- `eval`
- `document.write`

Trust Types - before / after

Without Trusted Types:

Unsafe content can be added to the DOM directly:

```
element.innerHTML = userInput; // Vulnerable
```

With Trusted Types:

*Create a Trusted Types policy to **sanitise** the content:*

```
const policy = TrustedTypes.createPolicy('default', {  
  createHTML: (input) => DOMPurify.sanitize(input),  
});  
element.innerHTML = policy.createHTML(userInput); // Safe
```

Trust Types - Benefits

Benefits

- Eliminates risky direct DOM manipulations.
- Forces developers to **sanitise input** or use safer alternatives like **textContent** (instead of **innerHTML**).

Key Takeaway

- Trusted Types transform how we handle dynamic HTML by **shifting security from guidelines to enforced rules**.
- Prevents DOM-based XSS at scale.

Securing Las Vegas Nightlife App

Challenge 4



<https://github.com/JavanXD/LasVegasNightlifeApp-Workshop>

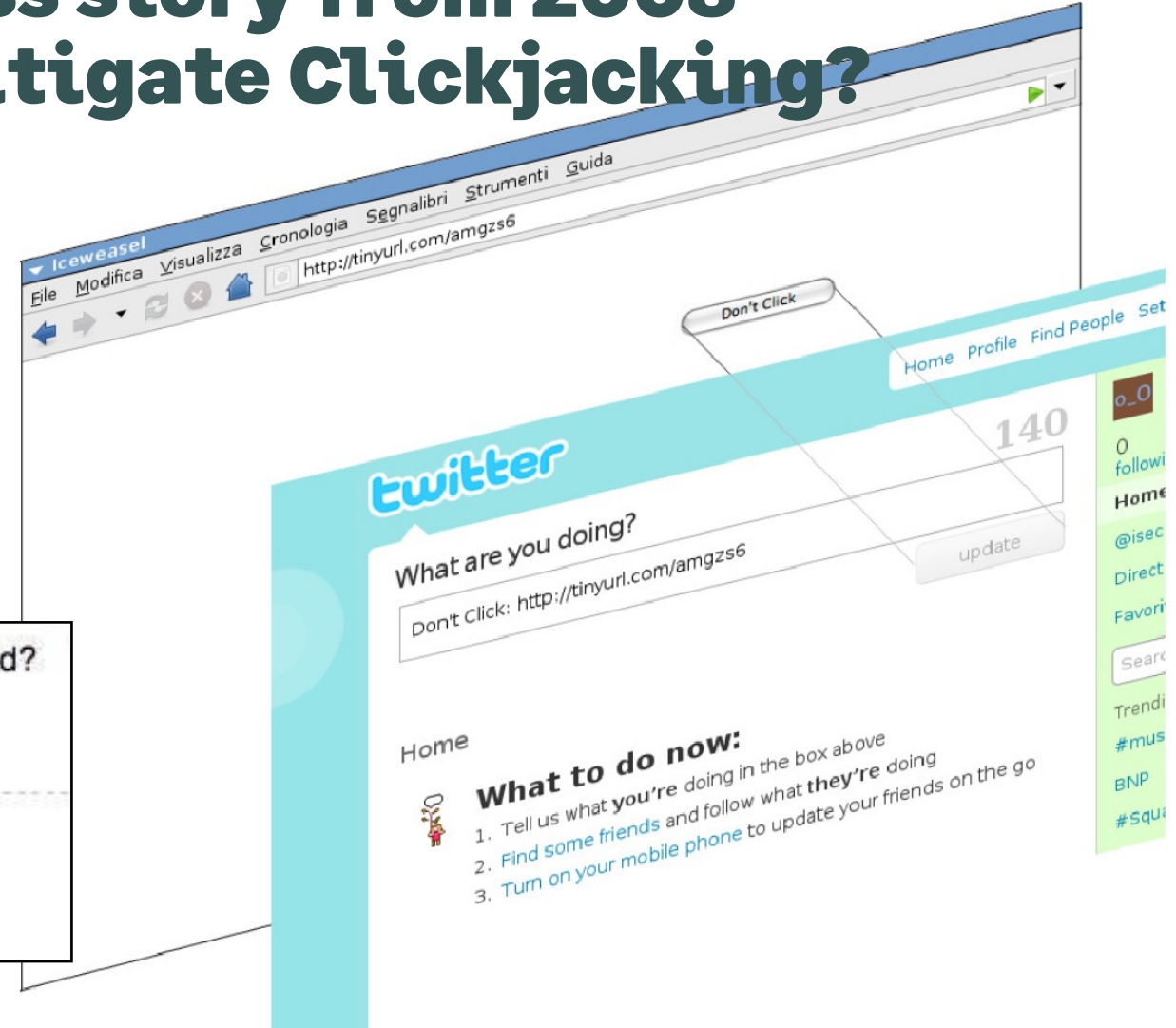
Anti-Clickjacking

X-Frame-Options: SAMEORIGIN / DENY

Or more modern:

Content-Security-Policy: frame-ancestors 'self';

X-Frame-Options, a success story from 2008 (Internet Explorer 8) to mitigate Clickjacking?



Example: Twitter (2009) Tweet Bomb

Securing Las Vegas Nightlife App

Challenge 5



<https://github.com/JavanXD/LasVegasNightlifeApp-Workshop>

Constant Evolving: More to Come

- **Integrity-Policy** is part of Subresource Integrity (SRI)
 - This will allow websites to ensure that all of their scripts are protected
 - Also comes with an `integrity` attribute option.

Browser compatibility

[Report problems with this compatibility data](#) • [View data on GitHub](#)

	Desktop					Mobile						
	Chrome	Edge	Firefox	Opera	Safari	Chrome Android	Firefox for Android	Opera Android	Safari on iOS	Samsung Internet	WebView Android	WebView on iOS
Integrity-Policy	✓ 138	✓ 138	✗ No	✗ No	✗ No	✓ 138	✗ No	✗ No	✗ No	✗ No	✓ 138	✗ No

Tip: you can click/tap on a cell for more information.

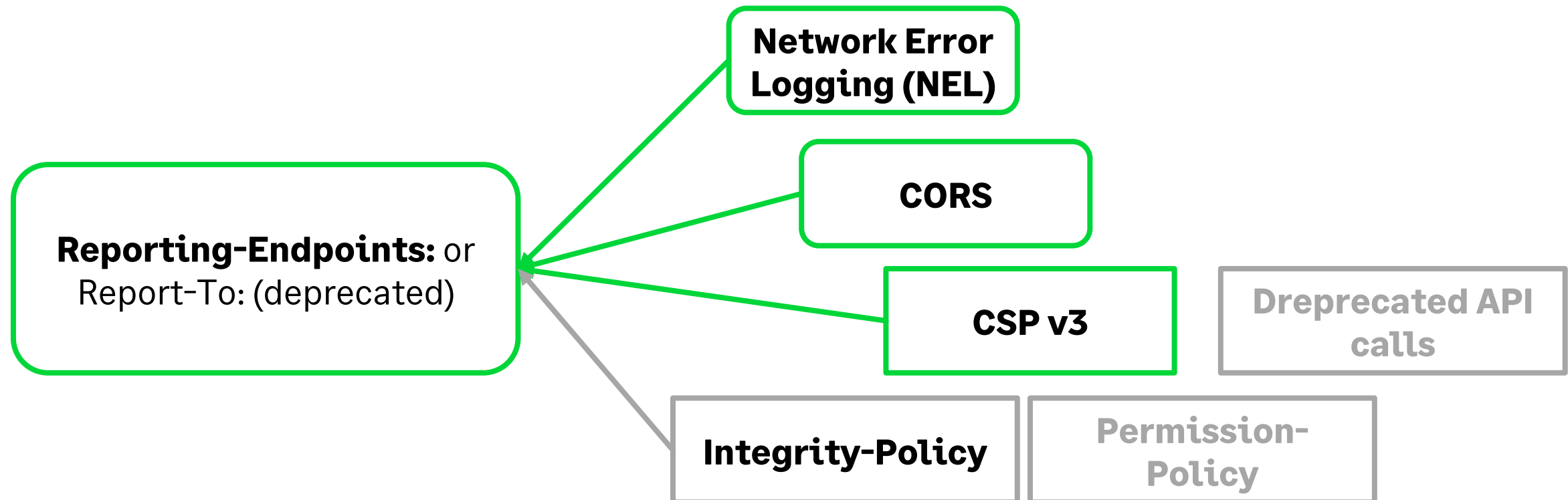
✓ Full support ✗ No support

See announcement here:

<https://attackanddefense.dev/2025/07/17/fire>

Reporting API - Response Header

- Collect reports with one defined header and be able to define one or multiple endpoints to receive those reports.



Securing Las Vegas Nightlife App

Challenge 6 & 7



<https://github.com/JavanXD/LasVegasNightlifeApp-Workshop>

Understanding COOP and COEP – Enabling Secure Isolation

CSRF Example – window.opener Abuse

A malicious page manipulates a popup's window.opener to perform actions on the trusted site.

```
// Malicious Page
const popup = window.open("https://trusted-site.com");
setTimeout(() => {
  popup.opener.location.href = "https://trusted-
site.com/transfer?amount=5000&to=attacker";
}, 2000);
```

What Are COOP and COEP?

- **Cross-Origin-Opener-Policy (COOP):**
 - Controls how the browser handles cross-origin interactions, **isolating the window or tab from others.**
 - Prevents cross-origin sharing of **window.opener** and other properties.
- **Cross-Origin-Embedder-Policy (COEP):**
 - Ensures that resources loaded by the page (e.g., images, scripts) are explicitly allowed by their origin to be embedded.

Why Do COOP and COEP Matter?

- **Security Benefits:**
 - Isolates the app to prevent cross-origin attacks like data leaks through `window.opener`.
 - Blocks **unauthorised embedding of resources**, mitigating supply chain attacks.
 - Provides a foundation for secure APIs like **SharedArrayBuffer**.
- **Enhanced Performance:**
 - Enables **cross-origin isolation**, allowing powerful browser features like `SharedArrayBuffer`.

COOP Header Values

COOP Values	Meaning	Use-Case
unsafe-none	Default, no isolation.	Legacy apps or those needing extensive cross-origin access.
same-origin	Isolates the browsing context to the same origin.	Apps requiring strict isolation and secure APIs.
same-origin-allow-popups	Similar to same-origin but allows popups.	Business apps with trusted popups, e.g., SSO flows.

COEP Header Values

COEP Values	Description	Use-Case
unsafe-none	Default, no restrictions.	Low-security apps or development environments.
require-corp	Requires resources to explicitly allow cross-origin embedding via Cross-Origin-Resource-Policy.	High-security apps needing isolation from supply chain risks.

Securing Las Vegas Nightlife App

Challenge 8



<https://github.com/JavanXD/LasVegasNightlifeApp-Workshop>

“Web security is increasingly an opt-in approach, leaving developers with both the opportunity and the responsibility to protect their applications.”

**Frederik Braun, Mozilla
at German OWASP Day 2024**

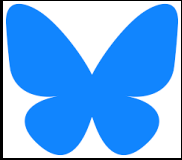
Takeaways

Takeaways

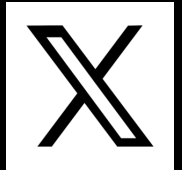
- **Shift from Reactive to Proactive Security**
- **Adopt Secure-by-Default Principles**
- **Leverage Platform Security Features**
- **Scale Security with Automation**
- **Commit to Bug Class Elimination**



[linkedin.com/in/javan-rasokat](https://www.linkedin.com/in/javan-rasokat)



<https://bsky.app/profile/javanrasokat.bsky.social>



[@javanrasokat](https://twitter.com/javanrasokat)



Connect on LinkedIn

Thank you!